Super-heterodyne Receiver

Presented To

Dr. Ahmed Hisham

T.A. Mohamed Khaled

Name	ID	Sec	BN
Kareem Ashraf Mostafa	91240568	3	3
Kareem Zakaria Abdelraouf	91240572	3	4

Table of Contents

Introduction **3**
Role of RF & IF **3**
Receiving 1st Station **4**
Normal Reception (with RF PBF) 4
Removing RF BPF 6
Local Oscillator Offset 8
Source Code **9**
AM Modulator 9
RF Stage 12
Local Oscillator Mixer 13
IF Stage 14
Baseband Detection 14
Appendix **15**

Introduction

Analog communication systems enable the transmission of information over long distances using modulation techniques. One of the most important receiver architectures is the Superheterodyne Receiver, which uses frequency conversion to process signals at a fixed intermediate frequency, improving selectivity and performance.

In this project, an analog communication system is simulated using MATLAB. Audio signals are used as baseband messages, modulated using DSB-SC, and combined through Frequency Division Multiplexing (FDM). At the receiver, the desired signal is recovered through RF filtering, mixing, IF filtering, and baseband detection.

This work demonstrates key concepts such as modulation, filtering, and frequency translation, while providing practical experience in implementing communication systems through simulation.

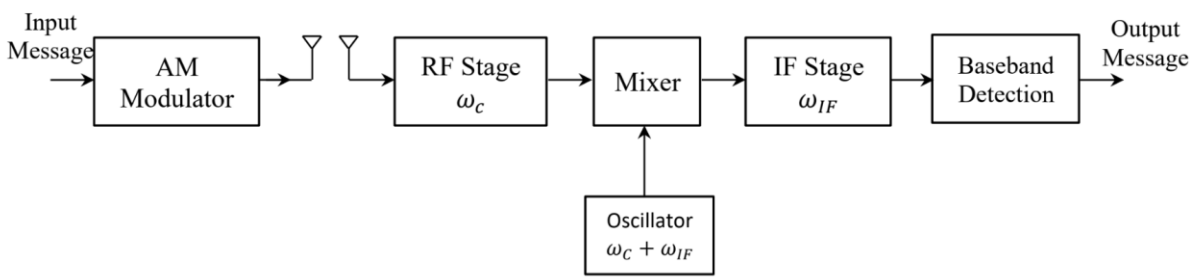

Figure 1: AM modulator and a super-heterodyne receiver

Role of RF & IF

The RF stage isolates the desired station from the FDM signal and rejects image frequencies and adjacent channels using a band-pass filter. The IF stage shifts the selected signal to a fixed intermediate frequency, which allows simpler and more selective filtering regardless of the original carrier frequency. Finally, the baseband detector down-converts the IF signal to the original audio frequency and recovers the message. The IF stage is necessary because it standardizes the frequency for filtering and amplification, improving selectivity and reducing receiver complexity.

Receiving 1st Station

The first station corresponds to an audio signal (*Short_FM9090.wav*), which is modulated using Double Sideband Suppressed Carrier (DSB-SC) with a carrier frequency of 100 kHz. The receiver is designed with an intermediate frequency (IF) of 15 kHz. The Frequency Division Multiplexing (FDM) signal consists of five modulated signals, with adjacent carrier frequencies spaced by 30 kHz. The following results are presented for different scenarios, including normal reception, reception without the RF band-pass filter (BPF), and reception with local oscillator (LO) frequency offsets of 0.1 kHz and 1 kHz. In the final stage, baseband detection is performed to down-convert the signal to baseband, followed by low-pass filtering (LPF) to recover the original message.

Normal Reception (with RF PBF)

In the normal reception scenario, the demodulated signal was reproduced using the MATLAB sound command, and the station was clearly audible with no observable distortion or image interference, confirming correct receiver operation and proper local oscillator alignment.

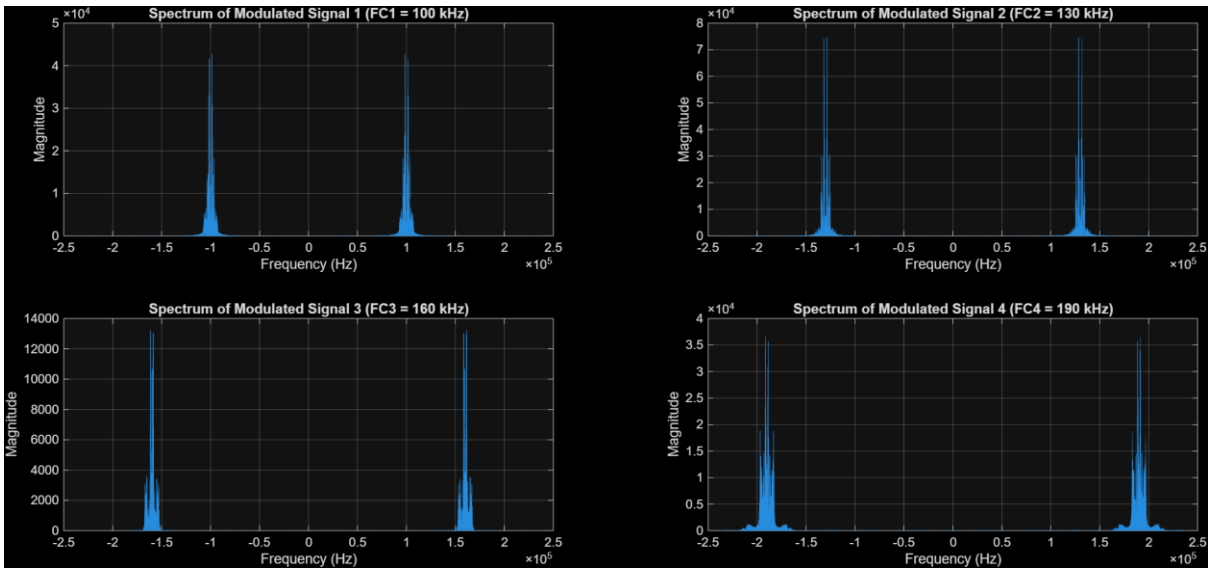

Figure 2: mod-signals-1-4

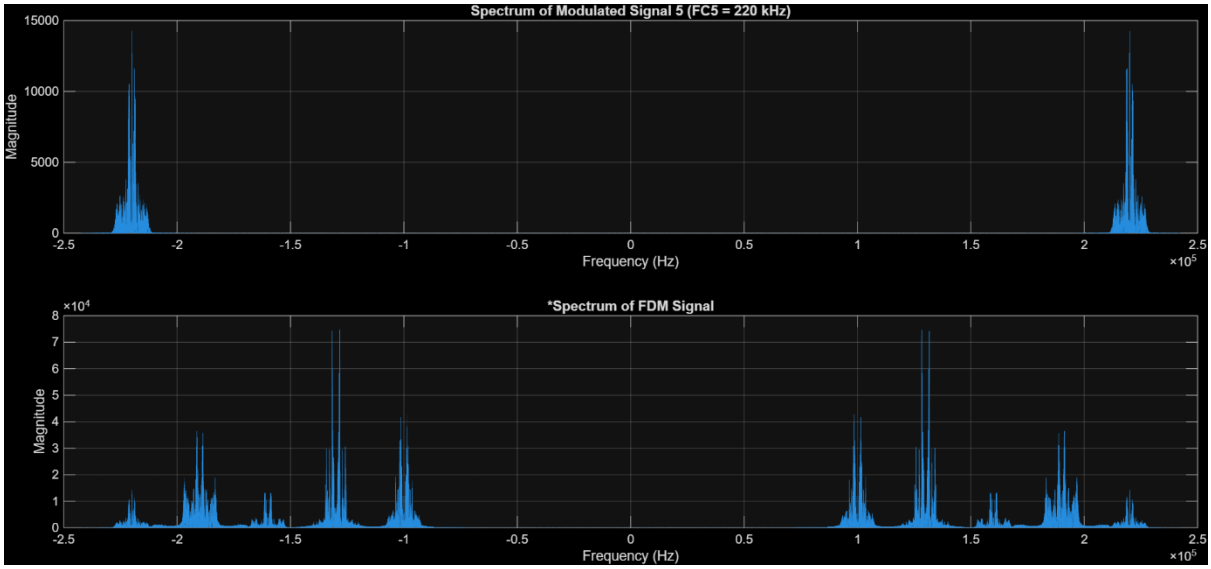

Figure 3: mod-signal5-and-FDM-spectrum

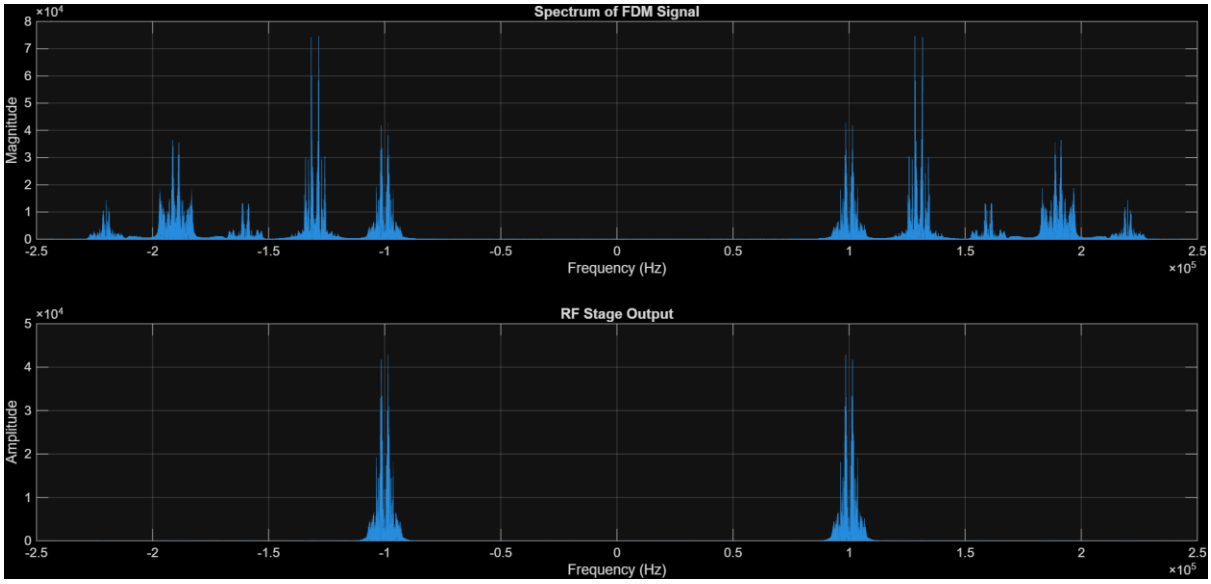

Figure 4: RF-output

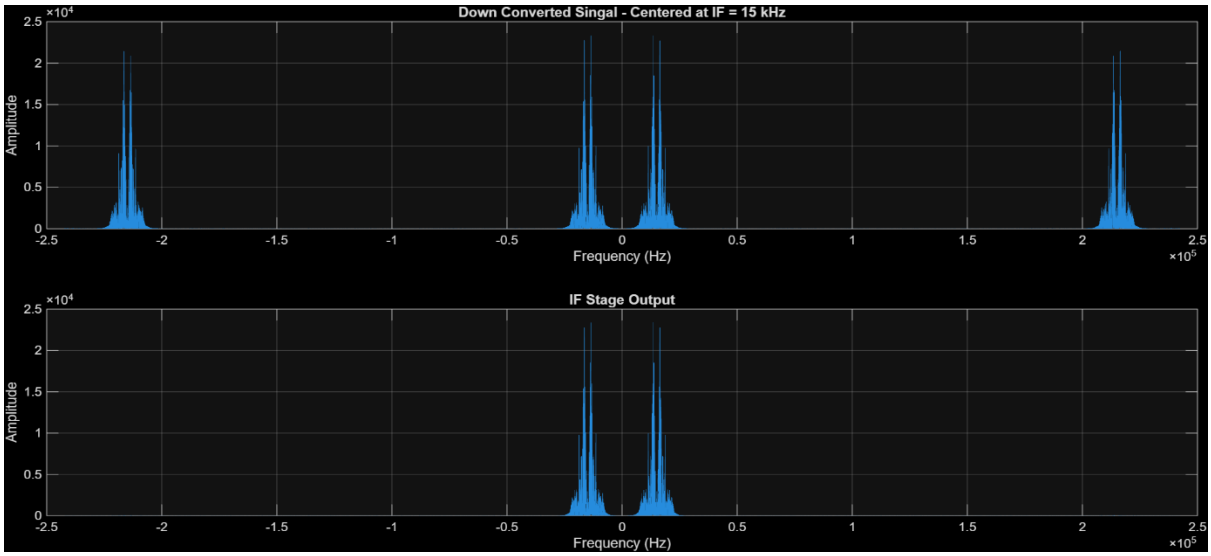

Figure 5: IF-output

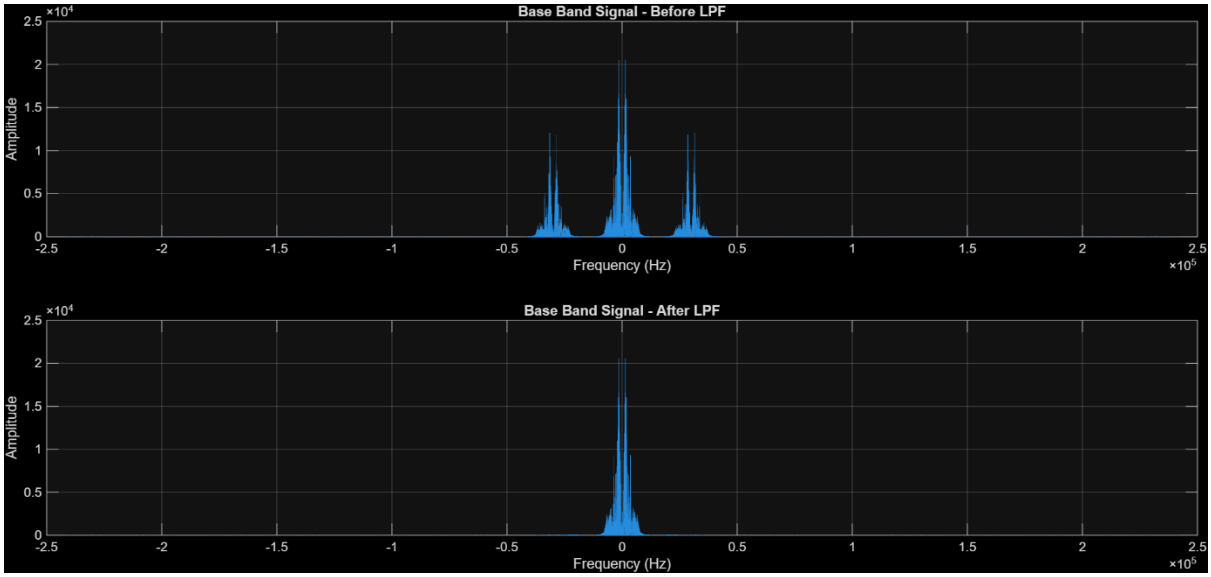

Figure 6: baseband-output.

Removing RF BPF

When the RF band-pass filter (BPF) is removed, the receiver loses its ability to reject image frequencies and adjacent channel interference. In this case, the image signal (Short_Quran_Palestine.wav) with a carrier frequency $F_{c2} = 130\text{ kHz} = 100\text{ kHz} + 2F_{IF}$is also down-converted to the same intermediate frequency. As a result, both signals overlap in the IF stage, producing noticeable interference and distortion, and significantly degrading the audio quality when reproduced using the MATLAB sound command.

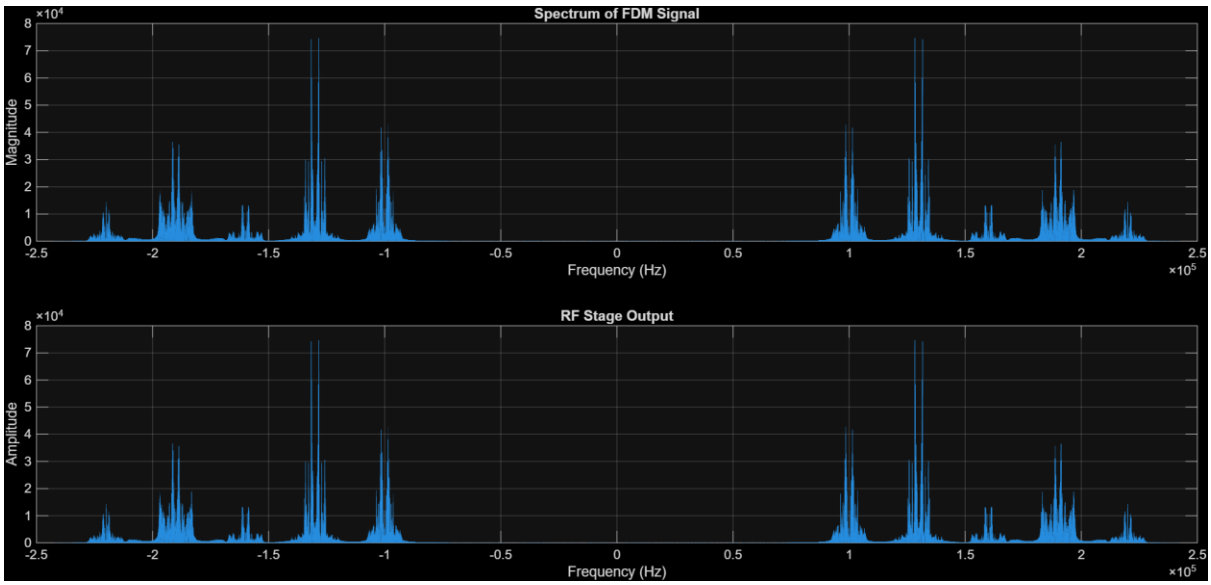

Figure 7: RF-output-without-BPF

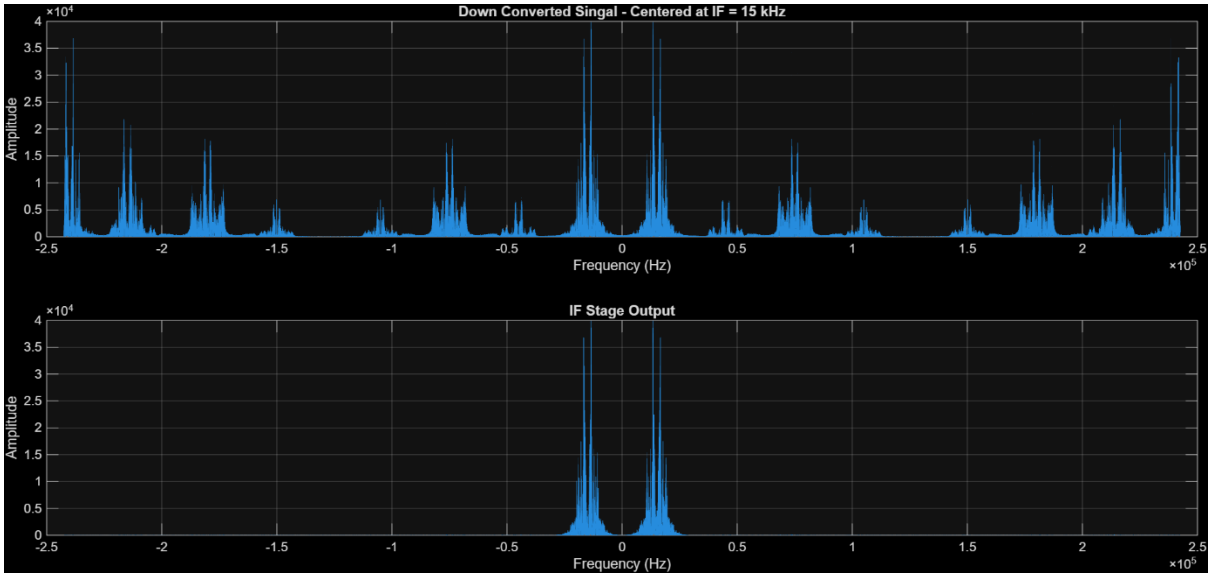

Figure 8: IF-output-no-RF-BPF

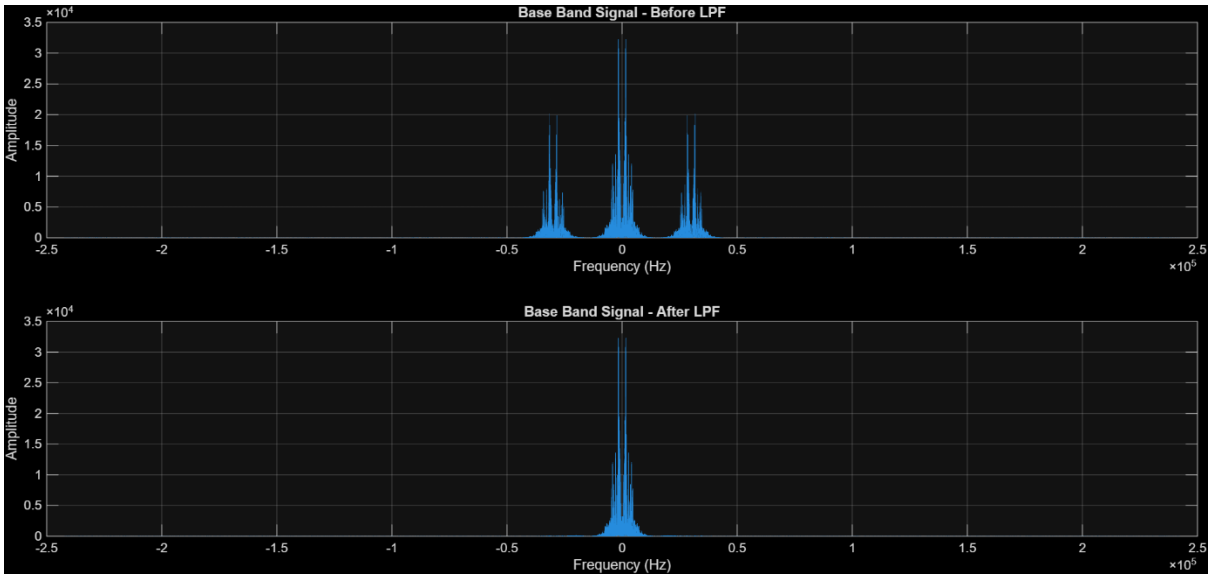

Figure 9: baseband-output-no-RF-BPF

Local Oscillator Offset

When the receiver oscillator has a frequency offset, the down-converted signal at the intermediate frequency (IF) is shifted by the same amount.

- **For a 0.1 kHz offset:** The frequency shift is small, so the demodulated audio remains largely intelligible. There is a slight pitch change, but the station can still be heard clearly using the MATLAB sound command. The spectrum shows a minor displacement of the signal components.
- **For a 1 kHz offset:** The frequency shift becomes significant, causing noticeable distortion in the audio. The demodulated station is heard with poor quality, exhibiting pitch errors and possible interference with adjacent channels. The spectrum shows a clear displacement of the signal components, reflecting the degraded sound quality.
- **Note that:** Signal isn't centered at 0Hz. It's centered at 1 kHz due to 1kHz Local Oscillator offset.

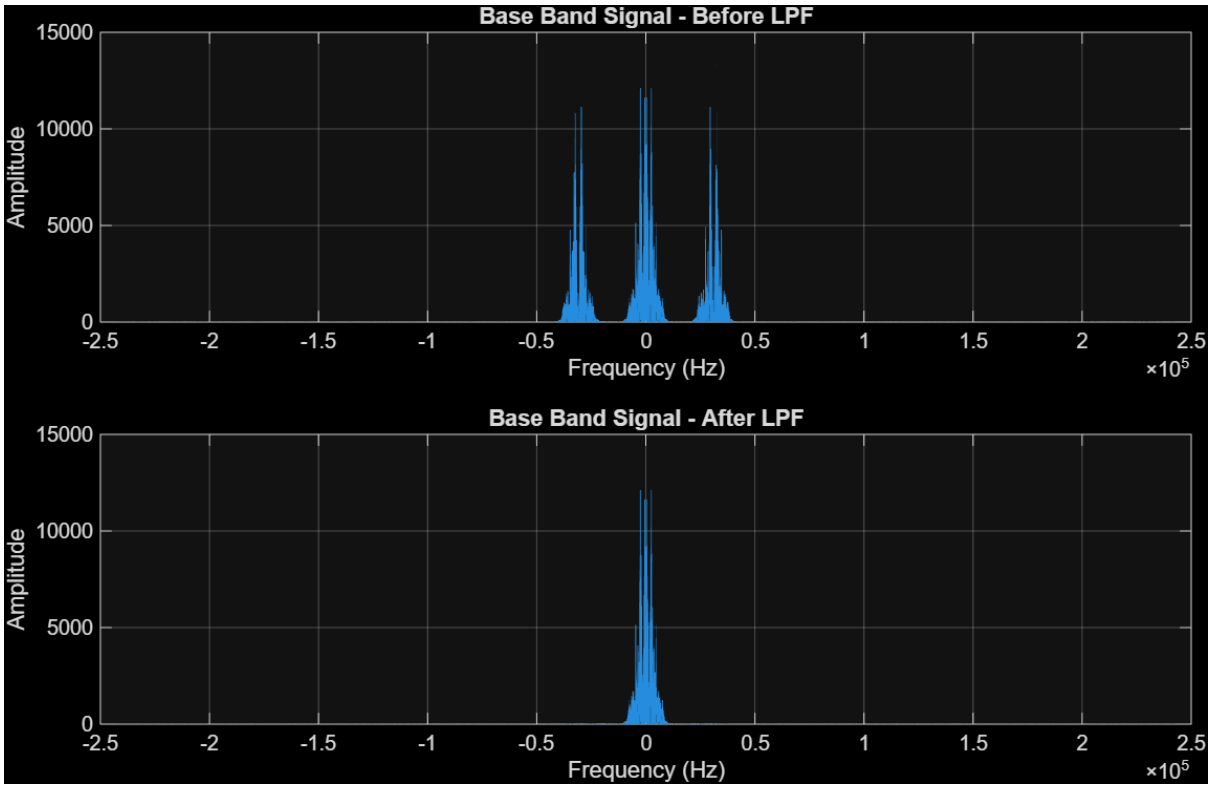

Figure 10: baseband-output-with-1kHz-LO-offset

Source Code

The following sections present the source code for each block of the system depicted in Figure 1.

AM Modulator

This block of code initializes the necessary parameters and prepares the input audio signals for modulation. The stereo audio files for five stations are read and converted to mono by combining the left and right channels. Since the signals have different lengths, zero-padding is applied to ensure all signals are of equal length. The signals are then up sampled to satisfy the Nyquist criterion for the chosen carrier frequencies. Time and frequency vectors are generated, and each signal is modulated using Double Sideband Suppressed Carrier (DSB-SC) onto its respective carrier frequency. Finally, the modulated signals are combined to form the Frequency Division Multiplexed (FDM) signal, which represents the transmitted multi-station analog signal.

```
% ==========================
% Needed Parameters
% ==========================
FC1 = 100e3;      % 1st Carrier Frequency in Hz.
FC2 = 130e3;      % 2nd Carrier Frequency in Hz.
FC3 = 160e3;      % 3rd Carrier Frequency in Hz.
FC4 = 190e3;      % 4th Carrier Frequency in Hz.
FC5 = 220e3;      % 5th Carrier Frequency in Hz.

% ==========================
% Read Audio Signals
% ==========================
[sig1_sterio, Fs1] = audioread('../assets/Short_FM9090.wav');        % 1st station
[sig2_sterio, Fs2] = audioread('../assets/Short_QuranPalestine.wav');  % 2nd station
[sig3_sterio, Fs3] = audioread('../assets/Short_BBCArabic2.wav');     % 3rd station
[sig4_sterio, Fs4] = audioread('../assets/Short_RussianVoice.wav');    % 4th station
[sig5_sterio, Fs5] = audioread('../assets/Short_SkyNewsArabia.wav');   % 5th station
```

```
% Mono Conversion -> as they are sterio signals
sig1_mono = sig1_sterio(:,1) + sig1_sterio(:,2);      % Single Row
sig2_mono = sig2_sterio(:,1) + sig2_sterio(:,2);      % Single Row
sig3_mono = sig3_sterio(:,1) + sig3_sterio(:,2);      % Single Row
sig4_mono = sig4_sterio(:,1) + sig4_sterio(:,2);      % Single Row
sig5_mono = sig5_sterio(:,1) + sig5_sterio(:,2);      % Single Row

% Zero Padding -> in case of unequal lengthes
len1 = length(sig1_mono);
len2 = length(sig2_mono);
len3 = length(sig3_mono);
len4 = length(sig4_mono);
len5 = length(sig5_mono);
lens = [len1 len2 len3 len4 len5];
[max_len, sig_idx] = max(lens);
if  sig_idx == 1       % sig1 is the longest
   sig2_mono = [sig2_mono; zeros(len1 - len2, 1)];
   sig3_mono = [sig3_mono; zeros(len1 - len3, 1)];
   sig4_mono = [sig4_mono; zeros(len1 - len4, 1)];
   sig5_mono = [sig5_mono; zeros(len1 - len5, 1)];
elseif sig_idx == 2     % sig2 is the longest
   sig1_mono = [sig1_mono; zeros(len2 - len1, 1)];
   sig3_mono = [sig3_mono; zeros(len2 - len3, 1)];
   sig4_mono = [sig4_mono; zeros(len2 - len4, 1)];
   sig5_mono = [sig5_mono; zeros(len2 - len5, 1)];
elseif sig_idx == 3     % sig3 is the longest
   sig1_mono = [sig1_mono; zeros(len3 - len1, 1)];
   sig2_mono = [sig2_mono; zeros(len3 - len2, 1)];
   sig4_mono = [sig4_mono; zeros(len3 - len4, 1)];
```

```
    sig5_mono = [sig5_mono; zeros(len3 - len5, 1)];
elseif sig_idx == 4     % sig4 is the longest
    sig1_mono = [sig1_mono; zeros(len4 - len1, 1)];
    sig2_mono = [sig2_mono; zeros(len4 - len2, 1)];
    sig3_mono = [sig3_mono; zeros(len4 - len3, 1)];
    sig5_mono = [sig5_mono; zeros(len4 - len5, 1)];
elseif sig_idx == 5     % sig5 is the longest
    sig1_mono = [sig1_mono; zeros(len5 - len1, 1)];
    sig2_mono = [sig2_mono; zeros(len5 - len2, 1)];
    sig3_mono = [sig3_mono; zeros(len5 - len3, 1)];
    sig4_mono = [sig4_mono; zeros(len5 - len4, 1)];
end

% Upsampling by 11 -> 44.1 * 11 > 2 * (100 + 4 * 30 + (BW/2 = 20/2)) "Nyquest Criteria"
upsamp_factor = 11;
sig1_sampled = interp(sig1_mono, upsamp_factor);
sig2_sampled = interp(sig2_mono, upsamp_factor);
sig3_sampled = interp(sig3_mono, upsamp_factor);
sig4_sampled = interp(sig4_mono, upsamp_factor);
sig5_sampled = interp(sig5_mono, upsamp_factor);
Fs_new = Fs1 * upsamp_factor;
Ts_new = 1 / Fs_new;


% ==========================
% Time & Frequency Vectors
% ==========================
N = length(sig1_sampled);
t = (0 : N-1) * Ts_new;          % time vector
f = (-N/2 : N/2-1) * (Fs_new / N);  % frequency vector
```

```
% ==========================
% AM Modulation using DSB-SC
% ==========================
carrier1 = cos(2 * pi * FC1 * t);      % Carrier for sig1_sampled
carrier2 = cos(2 * pi * FC2 * t);      % Carrier for sig2_sampled
carrier3 = cos(2 * pi * FC3 * t);      % Carrier for sig3_sampled
carrier4 = cos(2 * pi * FC4 * t);      % Carrier for sig4_sampled
carrier5 = cos(2 * pi * FC5 * t);      % Carrier for sig5_sampled

sig1_mod = sig1_sampled .* carrier1';
sig2_mod = sig2_sampled .* carrier2';
sig3_mod = sig3_sampled .* carrier3';
sig4_mod = sig4_sampled .* carrier4';
sig5_mod = sig5_sampled .* carrier5';

sig_FDM = sig1_mod + sig2_mod + sig3_mod + sig4_mod + sig5_mod;
```

RF Stage

This block implements the RF stage of the super-heterodyne receiver using a band-pass filter (BPF) to isolate the desired station from the FDM signal. The estimated bandwidth of the signal is 20 kHz, and each carrier is spaced by 30 kHz. A FIR band-pass filter is designed around the target carrier frequency, and the FDM signal is passed through this filter to suppress adjacent channels and image frequencies. The index n is tunable and can be explicitly set from 0 to 4 to select which signal to receive (signal 1 corresponds to n = 0, signal 2 to n = 1, and so on). Commenting out the filter line allows simulation of the scenario without RF filtering.

```
% ============================
% RF Stage - BPF
% ============================
BW = 20e3;        % estimated from signal plot
n = 0;            % signal index [0, 1, 2, 3, 4 == sig1, sig2, sig3, sig4, sig5]
deltaFC = 30e3;   % each signal carrier differs by 30 kHz
```

```
% BPF for signal 1+n - centered at FC1 + 30*n kHz
BPF_low = (FC1 + deltaFC*n - BW/2) / (Fs_new/2);    % lower cutoff frequency
BPF_upp = (FC1 + deltaFC*n + BW/2) / (Fs_new/2);    % upper cutoff frequency
BPF = fir1(200, [BPF_low, BPF_upp], 'bandpass');    % FIR BPF

% Pass FDM signal to RF BPF
RF_out = filter(BPF, 1,sig_FDM);
% RF_out = sig_FDM;   % Remove RF BPF
```

Local Oscillator Mixer

This block implements the mixer stage of the super-heterodyne receiver. The filtered RF signal is multiplied by a local oscillator (LO) signal to down-convert it to the intermediate frequency (IF). The LO frequency is set as $F_{LO} = F_C + IF$, where $F_C$is the carrier of the selected station and $IF = 15$ kHz. Optional frequency offsets can be introduced in the LO to simulate the effects of small mismatches (e.g., 0.1 kHz or 1 kHz) on the received signal. The output of this stage, IF_in, represents the signal centered around the IF, ready for further filtering.

```
% ============================
% Mixer - RF_out & Local Oscilator
% ============================
IF = 15e3;
FLO = IF + FC1 + deltaFC * n;
% FLO = IF + FC1 + deltaFC * n + 100;     % Frequency Offsit = 0.1 kHz
% FLO = IF + FC1 + deltaFC * n + 1000;    % Frequency Offsit = 1 kHz

% Local Oscillator Carrier
carrier_lo = cos(2 * pi * FLO * t);

% Down Convert signal to IF
IF_in = RF_out .* carrier_lo';
```

IF Stage

This block implements the intermediate frequency (IF) stage of the super-heterodyne receiver. A band-pass FIR filter is designed around the intermediate frequency ($IF = 15$ kHz) with a bandwidth of 20 kHz to isolate the desired signal while rejecting unwanted frequency components. The mixer output (IF_in) is passed through this IF BPF, producing IF_out, which contains the selected station centered at the intermediate frequency and is ready for baseband detection.

```
% ============================
% IF Stage - BPF
% ============================

% IF BPF - Centered at IF
IF_BPF_low = (IF - BW/2) / (Fs_new/2);    % lower cutoff frequency
IF_BPF_upp = (IF + BW/2) / (Fs_new/2);    % upper cutoff frequency
IF_BPF = fir1(200, [IF_BPF_low, IF_BPF_upp], 'bandpass');    % FIR BPF

% Pass Mixer Output to IF BPF
IF_out = filter(IF_BPF, 1,IF_in);
```

Baseband Detection

This block implements the baseband detection stage of the super-heterodyne receiver. The signal at the intermediate frequency (IF_out) is multiplied by a cosine at the IF to down-convert it to baseband. A low-pass FIR filter, with a cutoff equal to the signal bandwidth, is then applied to remove high-frequency components and recover the original audio signal. The filtered signal is resampled back to the original sampling frequency and played using the MATLAB sound command, allowing the recovered station to be audibly verified.

```
% ============================
% Base Band Detection
% ============================
sig_BB = IF_out .* cos(2 * pi * IF * t)';      % Down Convert signal to base band
```

```
% LPF - Cutoff Frequency = Signal BW
fc = BW;        % Filter cutoff
LPF = fir1(200, fc / (Fs_new/2));    % FIR LPF

sig_BB_filtered = filter(LPF, 1, sig_BB);

sig_out = resample(sig_BB_filtered,Fs1,Fs_new);
sound(sig_out, Fs1);
```

Appendix

→ GitHub Repo